

# Scalable Quantum Neighborhood Search for the Permutation Flow Shop Problem via Windowed QUBO Decomposition

Remigiusz Wojewódzki<sup>1</sup> and Wojciech Bożejko<sup>2</sup>

<sup>1</sup> Wrocław University of Science and Technology, Poland  
remigiusz.wojewodzki@gmail.com

<sup>2</sup> Wrocław University of Science and Technology, Poland  
wojciech.bozejko@pwr.edu.pl

---

**Abstract.** We extend the quantum neighborhood framework for the Permutation Flow Shop Scheduling Problem (PFSP) introduced in [?] by introducing a third class of neighborhood structures, *quantum QUBO enhanced*, designed to operate on larger benchmark instances than previously possible on real quantum annealing hardware. The original framework comprised eight neighborhoods: four classical (Adjacent, Fibonacci, Dynasearch, Motzkin) and four quantum QUBO counterparts executable on a D-Wave quantum processing unit (QPU) for instances with  $n = 20$  jobs. The new enhanced class introduces four additional neighborhoods that extend QPU applicability: (i) Quantum Adjacent Enhanced and Quantum Fibonacci Enhanced, which remove the delta-filtering heuristic and support instances up to  $n = 200$  and  $n = 500$  respectively, leveraging the linear ( $K = n - 1$ ) QUBO variable count of these neighborhoods; and (ii) Quantum Dynasearch Enhanced and Quantum Motzkin Enhanced, which introduce a windowed QUBO decomposition enabling execution on instances up to  $n = 50$ , where the full  $O(n^2)$ -variable QUBO would otherwise exceed QPU capacity. All twelve neighborhoods are evaluated on standard Taillard benchmark instances under Iterated Local Search and Simulated Annealing, measured by Relative Percentage Deviation (RPD) from best-known makespans across six time budgets (100–10 000 ms).

**Keywords:** Flow shop scheduling · QUBO · Quantum annealing · D-Wave · Neighborhood search · Windowed decomposition

## 1 Introduction

The Permutation Flow Shop Scheduling Problem (PFSP) is a classical NP-hard combinatorial problem [?] in which  $n$  jobs must be processed on  $m$  machines in identical sequence, with the objective of minimizing makespan  $C_{\max}$ . Neighborhood-based local search remains the dominant paradigm for obtaining high-quality solutions within practical time budgets.

In prior work [?], we formulated four classical PFSP neighborhood structures as Quadratic Unconstrained Binary Optimization (QUBO) problems and

executed them on D-Wave quantum annealing hardware, providing the first empirical comparison of classical and quantum neighborhood search under identical wall-clock time budgets. The experiments were limited to Taillard instances with  $n = 20$  jobs due to the variable-count constraints of the QUBO formulations and the effective embedding capacity of the D-Wave Advantage QPU.

The present work addresses two limitations of that study:

1. **No delta filter on QPU.** The original implementation filtered out non-improving swap candidates ( $\delta_k \geq 0$ ) before submitting the QUBO to the QPU, a heuristic necessary for the classical simulator but not for real QPU hardware, which handles the full variable space natively.
2. **Fixed instance size.** The  $O(n^2)$  variable count of Dynasearch and Motzkin neighborhoods limited experiments to  $n = 20$ , while Adjacent and Fibonacci ( $K = n - 1$  variables) could theoretically scale to much larger instances within QPU capacity.

We address both limitations by introducing the *quantum QUBO enhanced* class, comprising four new neighborhood formulations. The resulting twelve-neighborhood taxonomy — four classical, four quantum QUBO, four quantum QUBO enhanced — provides a structured framework for studying how quantum optimization scales with instance size and QUBO graph structure in a practically relevant combinatorial problem.

#### *Contributions.*

1. A windowed QUBO decomposition for Dynasearch and Motzkin neighborhoods, enabling QPU execution for  $n \leq 50$ .
2. Delta-filter-free QUBO formulations for Adjacent and Fibonacci neighborhoods, supporting  $n \leq 200$  and  $n \leq 500$  respectively on D-Wave Advantage.
3. The first empirical comparison of all three neighborhood classes across the full Taillard benchmark range ( $n \in \{20, 50, 100, 200, 500\}$ ) under identical metaheuristic frameworks and time budgets.
4. A quantitative analysis of the tractability boundary for each neighborhood class on current QPU hardware.

## 2 Problem Definition

*Permutation Flow Shop Problem.* Given a matrix of processing times  $P = [p_{k,j}]_{m \times n}$ , where  $k = 1, \dots, m$  denotes a machine and  $j = 1, \dots, n$  a job, a permutation  $\pi$  determines the order in which jobs are processed identically on all machines. The completion time  $C_{k,j}$  satisfies:

$$C_{1,j} = C_{1,j-1} + p_{1,\pi(j)}, \quad C_{k,1} = C_{k-1,1} + p_{k,\pi(1)}, \quad (1)$$

$$C_{k,j} = \max\{C_{k,j-1}, C_{k-1,j}\} + p_{k,\pi(j)}. \quad (2)$$

The objective is to find  $\pi^* = \arg \min_{\pi} C_{m,n}$ .

*Metaheuristics.* We employ Iterated Local Search (ILS) [?] with a tabu list (tenure  $\tau = 10$ ) and Simulated Annealing (SA) [?] with parameters  $T_{\text{init}} = 1000$ ,  $\alpha = 0.95$ , reheating factor  $r = 1.5$ , stagnation threshold 500 ms. Both algorithms are evaluated under identical time budgets.

*Diversification — Mushroom List.* ILS maintains an elite solution pool (*mushroom list*) of capacity  $k = 10$  to guide restarts when all candidate moves are tabu and the aspiration criterion is not met. The pool stores the  $k$  best distinct permutations encountered during search, ranked by  $C_{\text{max}}$ . On a diversification event, a *double-bridge* (4-opt) perturbation is applied to the least-recently-used elite: the permutation is split into four consecutive segments  $A \mid B \mid C \mid D$  at three uniformly sampled cut positions and reconnected as  $A \mid C \mid B \mid D$ . This move lies outside the 2-opt neighbourhood and cannot be undone by any single adjacent or endpoint swap, ensuring genuine escape from the current basin of attraction. The pool is updated after every improvement to  $C_{\text{max}}^*$ ; when the pool is empty (early search), the fallback is a uniform random restart. Round-robin selection across the pool ensures that successive diversification events explore distinct elite neighbourhoods rather than repeatedly perturbing the single global best.

### 3 Neighborhood Taxonomy

We organize the twelve neighborhoods into three classes.

#### 3.1 Classical Neighborhoods

The four classical neighborhoods are identical to those in [?,?]: Adjacent, Fibonacci, Dynasearch, and Motzkin. All use the Head+Tail technique for  $O(mn)$  delta evaluation and dynamic programming for optimal swap selection. Their QUBO formulations are recalled in Section 4.

#### 3.2 Block-Property Accelerators

All twelve neighborhoods apply the block-property accelerator of Smutnicki [?] to prune dominated swap candidates before search. The accelerator rests on a structural decomposition of the current permutation  $\pi$  into *blocks*: maximal contiguous sub-sequences in which the last machine  $m$  never idles. A position  $u \in \{1, \dots, n-1\}$  is a *block boundary* if

$$H[m-1][u] = H[m-2][u] + p_{m,\pi(u+1)}, \quad (3)$$

where  $H[k][j]$  denotes the completion time of job  $\pi(j)$  on machine  $k$  computed by the Head recurrence. The set of all block boundaries  $\mathcal{U} = \{u_1, \dots, u_q\} \subseteq \{1, \dots, n-1\}$  is computed in  $O(mn)$  from the Head matrix, which is available anyway for delta evaluation.

*Adjacent and Fibonacci accelerator.* For the adjacent-swap neighborhoods, swapping positions  $(a, a + 1)$  can improve  $C_{\max}$  only if  $a$  or  $a + 1$  coincides with a block boundary, i.e.,  $\{a, a + 1\} \cap \mathcal{U} \neq \emptyset$ . This filters up to  $n - 1$  swap candidates down to at most  $2(m - 1)$  on average, reducing evaluation cost from  $O(mn)$  to  $O(m \cdot \min(n, 2(m - 1)))$  per iteration. The Fibonacci neighborhood applies the same test, since it selects from the same adjacent-swap candidates.

*NPI accelerator for endpoint-swap neighborhoods.* For neighborhoods that consider non-adjacent endpoint-swap pairs  $(i, j)$  with  $j > i + 1$  (Dynasearch, Motzkin), a generalized *No-boundary-in-span* (NPI) filter applies: the pair  $(i, j)$  is pruned unless  $\mathcal{U} \cap [i, j] \neq \emptyset$ , i.e., at least one block boundary lies between positions  $i$  and  $j$ . The NPI filter reduces the  $O(n^2)$  candidate set to at most  $O(n \cdot (m - 1))$  pairs on average, matching the  $O(mn)$  scaling of delta computation. In the classical and quantum QUBO implementations, NPI is applied before the QUBO is assembled, so the number of QUBO variables is bounded by  $O(n(m - 1))$  rather than  $O(n^2)$ , which directly reduces QPU embedding overhead.

In all three neighborhood classes, block boundaries are computed once per move evaluation (reusing the Head matrix) and shared across all candidate pairs for that iteration.

### 3.3 Quantum QUBO Neighborhoods

The four quantum QUBO neighborhoods correspond directly to the classical structures reformulated as QUBO problems and solved on a D-Wave Advantage QPU via the `EmbeddingComposite` sampler. These are suitable for instances with  $n \leq 20$  and are described fully in [?].

### 3.4 Quantum QUBO Enhanced Neighborhoods

The four enhanced neighborhoods extend QPU applicability to larger instances through two mechanisms:

*Full QUBO without delta filtering (Adjacent, Fibonacci).* The original quantum QUBO implementation filtered candidate swaps with  $\delta_k \geq 0$  before QUBO construction to reduce the variable count for the classical simulator. On real QPU hardware this filter is unnecessary: the QPU naturally assigns  $x_k = 0$  to non-improving candidates. Removing it produces a complete QUBO that better reflects the true optimization landscape. Since  $K = n - 1$  for both neighborhoods, these formulations remain tractable on D-Wave Advantage ( $\sim 5,600$  physical qubits) up to  $n = 200$  (Adjacent, dense  $Q$ ) and  $n = 500$  (Fibonacci, tridiagonal  $Q$ ).

*Windowed QUBO decomposition (Dynasearch, Motzkin).* For Dynasearch and Motzkin, the full QUBO has  $K = O(n^2)$  variables with  $O(n^4)$  interactions, exceeding QPU capacity for  $n > 20$ . We introduce a windowed decomposition: the permutation  $\pi$  is partitioned into overlapping windows of size  $w$ , and a

**Table 1.** Tractability of neighborhood classes on D-Wave Advantage.  $K$  = QUBO variable count;  $n_{\max}$  = largest feasible instance.

| Class                 | Neighborhood $K$ |                       | $n_{\max}$     |
|-----------------------|------------------|-----------------------|----------------|
| Classical             | Adjacent         | $n - 1$               | $\infty$ (CPU) |
|                       | Fibonacci        | $n - 1$               | $\infty$ (CPU) |
|                       | Dynasearch       | $O(n^2)$              | $\infty$ (CPU) |
|                       | Motzkin          | $O(n^2)$              | $\infty$ (CPU) |
| Quantum QUBO          | Adjacent         | $n - 1$               | 20             |
|                       | Fibonacci        | $n - 1$               | 20             |
|                       | Dynasearch       | $O(n^2)$              | 20             |
|                       | Motzkin          | $O(n^2)$              | 20             |
| Quantum QUBO Enhanced | Adjacent         | $n - 1$               | 200            |
|                       | Fibonacci        | $n - 1$               | 500            |
|                       | Dynasearch       | $\leq 171$ (windowed) | 50             |
|                       | Motzkin          | $\leq 171$ (windowed) | 50             |

separate QUBO is built and solved independently for each window. Results from all windows are merged with greedy conflict resolution.

Formally, for window  $[s, s + w)$ , the QUBO is defined over the  $\binom{w}{2}$  candidate swap pairs  $(i, j)$  with  $s \leq i < j < s + w$ . Windows advance by step  $\lfloor w(1 - \rho) \rfloor$ , where  $\rho$  is the overlap ratio (default  $\rho = 0.5$ ). The window size  $w$  is chosen automatically to keep  $K(w) = w(w - 1)/2 \leq C_{\text{eff}}$ , where  $C_{\text{eff}} = 180$  is a conservative estimate of the effective D-Wave variable capacity for dense QUBOs:

$$w_{\max} = \left\lfloor \frac{1 + \sqrt{1 + 8C_{\text{eff}}}}{2} \right\rfloor = 19. \quad (4)$$

This enables Dynasearch and Motzkin Enhanced to execute on instances up to  $n = 50$ .

Table 1 summarizes the tractability analysis for all twelve neighborhoods.

## 4 QUBO Formulations

All neighborhoods share the general QUBO form:

$$\min_{\mathbf{x} \in \{0,1\}^K} \mathbf{x}^\top Q \mathbf{x} \xrightarrow{x_i = (1 - Z_i)/2} H_C = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j. \quad (5)$$

Diagonal entries  $Q_{kk} = \delta_k$  encode the makespan change from swap  $k$ , computed via the Head+Tail technique. Off-diagonal entries  $Q_{kl} = P$  penalize conflicting swap pairs, with  $P = \sum_k |\delta_k| + 1$  ensuring constraint dominance.

*Adjacent.* One-hot QUBO:  $Q_{ii} = \delta_i - P$ ,  $Q_{ij} = 2P$  (all pairs), enforcing selection of exactly one swap.

*Fibonacci.* Tridiagonal QUBO:  $Q_{ii} = \delta_i$ ,  $Q_{i,i+1} = P$ ; only  $n - 2$  nonzero off-diagonal entries. The number of feasible swap sets equals the Fibonacci number  $F_{n+1}$ .

*Dynasearch.*  $Q_{kl} = P$  iff segments  $(i_1, j_1)$  and  $(i_2, j_2)$  overlap:  $\max(i_1, i_2) \leq \min(j_1, j_2)$ . Nesting is forbidden.

*Motzkin.*  $Q_{kl} = P$  iff pairs conflict under Motzkin rules: shared endpoint or crossing arcs. Nesting and disjoint pairs are unconstrained ( $Q_{kl} = 0$ ). The number of feasible swap sets equals the Motzkin number  $M_n$  [?].

## 5 Experimental Setup

*Benchmarks.* We use standard Taillard benchmark instances [?]:  $n \in \{20, 50, 100, 200, 500\}$ ,  $m \in \{5, 10, 20\}$ . For each  $(n, m)$  configuration, 10 instances are used. Time budgets:  $t_{\max} \in \{100, 500, 1000, 2000, 5000, 10\,000\}$  ms. Seeds are fixed for reproducibility.

*Hardware.* Classical neighborhoods run on an Apple M4 Pro (24 GB RAM). Quantum QUBO and Enhanced neighborhoods use the D-Wave Advantage QPU via the Leap cloud platform, with `EmbeddingComposite` sampler and `num_reads = 100`.

*Metrics.* Solution quality is measured by Relative Percentage Deviation from the best-known Taillard makespan:

$$\text{RPD} = \frac{C_{\max} - C^*}{C^*} \times 100\%. \quad (6)$$

*Instance constraints.* Following the tractability analysis in Table 1:

- Quantum QUBO (original):  $n = 20$  only.
- Quantum Adjacent Enhanced:  $n \leq 200$ .
- Quantum Fibonacci Enhanced:  $n \leq 500$ .
- Quantum Dynasearch/Motzkin Enhanced:  $n \leq 50$ , window size  $w = 19$  (auto), overlap  $\rho = 0.5$ .

## 6 Results and Discussion

*Comparison at  $n = 20$ .* Table 2 compares all twelve neighborhoods on the smallest common instance size. The quantum QUBO enhanced variants are expected to perform comparably to the original quantum QUBO variants at  $n = 20$ , since the only difference is the removal of the delta filter — a change that affects the QUBO landscape but not the variable count or embedding structure. Any observed difference will quantify the effect of the filter on solution quality.

**Table 2.** Average RPD [%] for all twelve neighborhoods,  $t_{\max} = 1000$  ms,  $n = 20$ ,  $m = 5$ . **[PLACEHOLDER — to be filled with experimental results]**

| Class            | Neighborhood | ILS          | SA            |               | SA |
|------------------|--------------|--------------|---------------|---------------|----|
|                  |              | $t = 100$ ms | $t = 1000$ ms | $t = 1000$ ms |    |
| Classical        | Adjacent     | —            | —             | —             | —  |
|                  | Fibonacci    | —            | —             | —             | —  |
|                  | Dynasearch   | —            | —             | —             | —  |
|                  | Motzkin      | —            | —             | —             | —  |
| Quantum QUBO     | Adjacent     | —            | —             | —             | —  |
|                  | Fibonacci    | —            | —             | —             | —  |
|                  | Dynasearch   | —            | —             | —             | —  |
|                  | Motzkin      | —            | —             | —             | —  |
| Quantum Enhanced | Adjacent     | —            | —             | —             | —  |
|                  | Fibonacci    | —            | —             | —             | —  |
|                  | Dynasearch   | —            | —             | —             | —  |
|                  | Motzkin      | —            | —             | —             | —  |

**Table 3.** Average RPD [%] for quantum QUBO enhanced neighborhoods across instance sizes, ILS,  $t_{\max} = 5000$  ms. **[PLACEHOLDER]**

| Neighborhood         | $n = 20$ | $n = 50$ | $n = 100$ | $n = 200$ | $n = 500$ |
|----------------------|----------|----------|-----------|-----------|-----------|
| Classical Adjacent   | —        | —        | —         | —         | —         |
| Classical Fibonacci  | —        | —        | —         | —         | —         |
| Q-Adjacent Enhanced  | —        | —        | —         | n/a       | n/a       |
| Q-Fibonacci Enhanced | —        | —        | —         | —         | —         |
| Q-Dynasearch Enh.    | —        | —        | n/a       | n/a       | n/a       |
| Q-Motzkin Enh.       | —        | —        | n/a       | n/a       | n/a       |

*Scaling behavior.* Table 3 examines how the enhanced quantum neighborhoods scale with  $n$ . The primary hypothesis is that Quantum Fibonacci Enhanced, with its tridiagonal QUBO structure and linear variable count, will maintain competitive RPD relative to classical Fibonacci across all instance sizes, while Quantum Adjacent Enhanced will show increasing RPD for  $n > 100$  due to its dense QUBO graph requiring more physical qubits and deeper embedding chains.

*Windowed decomposition quality.* The windowed approach for Dynasearch and Motzkin Enhanced introduces suboptimality relative to the full QUBO: swaps crossing window boundaries are not considered. The magnitude of this quality loss will be measured by comparing Enhanced RPD against classical Dynasearch/Motzkin at  $n = 20$  (where both formulations are feasible) and quantified as a function of window overlap ratio  $\rho$ .

*Throughput analysis.* A key factor in quantum neighborhood performance is the number of neighborhood evaluations per second achievable within a fixed time budget. The enhanced formulations submit more QUBO calls per iteration (one per window) compared to the original formulations (one per iteration). The tradeoff between larger effective neighborhood coverage and increased QPU latency will be quantified.

## 7 Conclusions

We introduced the quantum QUBO enhanced neighborhood class, extending quantum annealing-based PFSP neighborhood search to larger instances through two complementary mechanisms: delta-filter removal for linearly-scaling QUBO structures (Adjacent, Fibonacci) and windowed QUBO decomposition for quadratically-scaling structures (Dynasearch, Motzkin).

The resulting twelve-neighborhood taxonomy provides a systematic framework for studying the interaction between QUBO graph structure, instance size, and metaheuristic performance in quantum combinatorial optimization. Preliminary analysis suggests that the tractability boundary varies substantially across neighborhoods: Fibonacci Enhanced remains feasible up to  $n = 500$  on current D-Wave hardware, while Motzkin and Dynasearch Enhanced are limited to  $n = 50$  by the windowed decomposition constraint.

*Future work.* Directions include: adaptive window sizing based on convergence feedback, integration of QAOA circuits on gate-based hardware (Willow processor), and investigation of alternative decomposition strategies that preserve cross-window swap interactions.

---

## References